

Ejercicio 3 - Arreglos dinámicos, ArrayList, List y Excepciones

Sistema de Control de Órdenes de Servicio de un Taller Automotriz

Objetivo

- Aplicar arreglos dinámicos para almacenar objetos dentro de un programa.
- Utilizar correctamente List y ArrayList para administrar colecciones de elementos.
- Agregar, consultar, modificar y eliminar objetos almacenados dinámicamente.
- Recorrer colecciones utilizando estructuras repetitivas.
- Realizar búsquedas dentro de una colección de objetos.
- Implementar manejo de excepciones utilizando try-catch.
- Comprender y aplicar el bloque finally dentro del manejo de excepciones.
- Aplicar correctamente los modificadores de visibilidad para mantener el encapsulamiento.
- Modelar el programa utilizando UML.
- Crear un programa principal que resuelva el problema utilizando programación orientada a objetos.

Introducción

Un taller automotriz desea implementar un sistema para administrar las órdenes de servicio correspondientes a los vehículos que ingresan para mantenimiento o reparación.

Durante una jornada, la cantidad de vehículos atendidos puede variar considerablemente. Algunos clientes solicitan nuevos servicios, otros necesitan realizar modificaciones a una orden previamente registrada y algunas órdenes pueden ser canceladas antes de iniciar el trabajo.

Debido a que no se conoce previamente la cantidad de órdenes que serán registradas, el sistema no deberá establecer un límite fijo. Las órdenes de servicio deberán almacenarse utilizando estructuras dinámicas mediante List y ArrayList.

Además, durante la utilización del sistema pueden producirse situaciones inesperadas, como ingresar datos incorrectos o intentar consultar, modificar o eliminar una orden inexistente. Estas situaciones deberán manejarse utilizando excepciones mediante try-catch y finally, evitando que el programa finalice inesperadamente.

Para administrar las órdenes de servicio se debe realizar lo siguiente:

1. Cada orden de servicio deberá almacenar como mínimo:
 - a. Número de orden.
 - b. Nombre del propietario.
 - c. Placa del vehículo.
 - d. Descripción del servicio.
 - e. Costo estimado.
2. El sistema deberá permitir registrar una cantidad dinámica de órdenes de servicio. No

deberá establecerse un límite fijo.

3. Cada nueva orden deberá agregarse a una colección implementada utilizando List y ArrayList.
4. Al registrar una orden se deberá validar que:
 - a. El número de orden no se encuentre registrado previamente.
 - b. El nombre del propietario no esté vacío.
 - c. La placa del vehículo no esté vacía.
 - d. La descripción del servicio no esté vacía.
 - e. El costo estimado sea mayor que 0.
5. El sistema deberá permitir consultar todas las órdenes de servicio registradas actualmente.
6. El sistema deberá permitir buscar una orden utilizando su número de orden.
7. Cuando se realice una búsqueda:
 - a. Si la orden existe, se deberá mostrar toda su información.
 - b. Si la orden no existe, el programa deberá informar al usuario sin finalizar inesperadamente su ejecución.
8. El sistema deberá permitir modificar una orden previamente registrada. Como mínimo se podrá modificar:
 - a. Descripción del servicio.
 - b. Costo estimado.
9. El sistema deberá permitir cancelar una orden de servicio. Cuando una orden sea cancelada, el objeto correspondiente deberá eliminarse de la colección dinámica.
10. El sistema deberá permitir consultar todas las órdenes asociadas a una placa específica. Si un mismo vehículo posee más de una orden registrada, deberán mostrarse todas las coincidencias encontradas.
11. El sistema deberá calcular el valor total de las órdenes activas, sumando los costos estimados de todas las órdenes almacenadas actualmente.
12. El sistema deberá calcular el costo promedio de las órdenes activas.
13. El sistema deberá determinar cuál es la orden que posee el costo estimado más alto.
14. El sistema deberá indicar cuántas órdenes se encuentran registradas actualmente.
15. El programa deberá utilizar manejo de excepciones mediante try-catch para controlar errores relacionados con entradas incorrectas del usuario, por ejemplo, cuando se espera ingresar un costo numérico y se proporciona un valor que no puede ser procesado correctamente.
16. El programa deberá utilizar try-catch para controlar al menos una operación relacionada con la búsqueda, modificación o cancelación de una orden que no se encuentre registrada.

17. Se deberá utilizar un bloque finally en al menos uno de los procesos que utilice manejo de excepciones. Este bloque deberá ejecutar una acción independientemente de si la operación fue realizada correctamente o produjo una excepción.
18. Después de controlar una excepción, el sistema deberá continuar funcionando y permitir al usuario regresar al menú principal.

Ejercicio

Análisis

Responda las siguientes preguntas:

1. ¿Qué clases son necesarias para representar el problema planteado?
2. ¿Qué propiedades y métodos tendrá cada clase?
3. ¿Qué tipo deben tener las propiedades y métodos de cada clase?
4. ¿Qué información deberá almacenarse dentro de la colección dinámica?
5. ¿Dónde utilizará List y dónde realizará la creación mediante ArrayList?
6. ¿Cuáles deben ser los modificadores de visibilidad de los miembros de cada clase?
7. ¿Qué parámetros serán requeridos por los métodos de sus clases?
8. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?
9. ¿Cómo identificará una orden específica dentro de la colección?
10. ¿Cómo agregará y eliminará órdenes de la colección dinámica?
11. ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?
12. ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?
13. ¿Cómo determinará cuál es la orden con el costo estimado más alto?
14. ¿Qué situaciones dentro del programa pueden producir una excepción?
15. ¿Qué operaciones serán protegidas utilizando try-catch?
16. ¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación?

Diseño

Desarrolle un diagrama de clases que muestre los miembros de cada clase y que establezca las relaciones necesarias entre ellas.

Incluya en su diagrama al driver program, que contiene el método main().

El diagrama deberá representar correctamente los atributos, métodos, tipos de datos, parámetros y modificadores de visibilidad identificados durante el análisis.

La colección dinámica utilizada para almacenar los objetos deberá estar representada correctamente dentro del diseño.

Use anotaciones (recuadros de texto) en su diagrama para aclarar cualquier información que considere necesaria, especialmente sobre el uso de la colección dinámica y el manejo de excepciones.

necesaria.

Programa

Desarrolle un sistema que contenga un driver program y las instancias necesarias de las clases identificadas durante su análisis.

El programa deberá utilizar List y ArrayList para almacenar dinámicamente las órdenes de servicio registradas.

Así mismo, debe mostrar un menú con las siguientes opciones:

1. Registrar orden: solicita la información necesaria y agrega una nueva orden de servicio a la colección.
2. Consultar órdenes: muestra todas las órdenes registradas actualmente.
3. Buscar orden: solicita el número de orden y muestra su información.
4. Modificar orden: permite modificar la información de una orden previamente registrada.
5. Cancelar orden: solicita el número de orden y elimina de la colección la orden correspondiente.
6. Consultar órdenes por placa: solicita la placa de un vehículo y muestra todas las órdenes asociadas a ella.
7. Reporte de costos: muestra el valor total y el costo promedio de todas las órdenes registradas actualmente.
8. Orden de mayor costo: muestra la información de la orden cuyo costo estimado sea el más alto.
9. Cantidad de órdenes: muestra cuántas órdenes de servicio se encuentran almacenadas actualmente.
10. Salir: termina la ejecución del programa.

El programa deberá utilizar correctamente operaciones de una colección dinámica para agregar, recorrer, buscar, modificar y eliminar objetos.

Las operaciones indicadas deberán manejar posibles errores mediante try-catch, evitando que el programa finalice inesperadamente.

Se deberá utilizar al menos un bloque finally para ejecutar una acción independientemente de si la operación asociada finalizó correctamente o produjo una excepción.

Material a entregar en Canvas

- **Archivo PDF con el análisis y diseño.**
 - No incluya únicamente la URL del diagrama; adjunte el diagrama dentro del PDF o como un archivo externo.
 - Use la plantilla proporcionada.
- **URL al repositorio público en GitHub** conteniendo todos los archivos .java necesarios para ejecutar el programa.

Evaluación

- **Análisis** (30 puntos)
 - **(5 puntos)** Se listan correctamente todos los requisitos funcionales del sistema a desarrollar, punto por punto.
 - **(5 puntos)** Se describe el propósito de cada una de las clases identificadas. La descripción es lógica y coherente con el problema presentado.
 - **(10 puntos)** Se describe correctamente el propósito de los atributos de cada clase y de la colección dinámica utilizada para almacenar los objetos.
 - **(10 puntos)** Se describe el propósito de cada método, los parámetros necesarios, los resultados producidos y las situaciones que requieren manejo de excepciones.
- **Diseño** (30 puntos)
 - **(8 puntos)** La representación de las clases coincide con el estándar del lenguaje UML.
 - **(12 puntos)** Están representados correctamente los atributos, métodos, tipos de datos, parámetros, modificadores de visibilidad y la colección dinámica, coincidiendo con lo descrito en el análisis.
 - **(10 puntos)** Las relaciones entre las clases están correctamente representadas.
- **Programa** (40 puntos)
 - **(10 puntos)** La cantidad de clases identificadas es suficiente para resolver el problema planteado y el sistema utiliza correctamente programación orientada a objetos y encapsulamiento.
 - **(15 puntos)** Se utilizan correctamente List y ArrayList para almacenar objetos dinámicamente y se implementan las operaciones necesarias para agregar, recorrer, buscar, modificar y eliminar elementos.
 - **(10 puntos)** Se implementa correctamente el manejo de excepciones mediante try-catch, evitando que entradas u operaciones inválidas finalicen inesperadamente la ejecución del programa.
 - **(5 puntos)** Se utiliza correctamente al menos un bloque finally y el programa continúa funcionando adecuadamente después del manejo de una excepción.